

Piano — CRM Harness per "Demo per Rome Future Week"

Stato: bozza di pianificazione (analisi + piano, non codice). Demo di riferimento: [~/Downloads/Demo per Rome Future Week.pdf](#) (5 atti). Canale messaggistica: **Telegram** (non WhatsApp). Parte telefonica: **esterna**, comunica via **webhook** con l'harness.

1. Quadro demo vs capability esistenti

Atto	Cosa chiede la demo	Stato nel repo
1	Chiamata in ingresso → assistente risponde, cattura consenso + preferenza contatto + dati persona	Manca webhook telefonico ; nessun campo preferenza/opt-in
2	Copilot analizza pubblico (storico, interessi, conversazioni, acquisti, preferenze, segmenti)	Coperto (skill <code>crm</code> + segmenti + interazioni)
3	Orchestrazione: Telegram a chi sceglie Telegram, email a chi sceglie email; agente marketing	Metà : campagne email (SES) esistono; manca trasporto Telegram + routing per canale
4	Cliente riceve messaggio Telegram, conversa, acquista (link)	Base (ingestione eventi Purchase); manca webhook Telegram conversazione
5	Richiama, assistente lo riconosce e sa dello stato ordine/corriere	Manca modello commerce + identità per numero + memoria cross-canale

Capability già presenti (verificato sul codice)

- CRM in DuckDB: `people`, `company`, `interaction`, `segment`, `campaign`, `campaign_send` (plus `email_thread`/`email_message`/`calendar_event`).

- Copilot = chat con l'agente + skill `crm` (`database-crm-system`): query DuckDB, report `.report.json`, segmenti, campagne.
 - Segmentazione: builder demografico + condizioni eventi (`apps/web/app/components/crm/segments/`, `apps/web/lib/segments.ts`).
 - Campagne email (AWS SES): coda, stati, retry, webhook SNS bounce/complaint (`apps/web/lib/campaigns.ts`).
 - Ingestione eventi CDP (`interaction`, type `Purchase`) + web tracker anon→identificato (`apps/web/app/api/crm/events/route.ts`, `/api/events/collect`, shadow+merge).
 - **Canale Telegram outbound: implementato nel runtime openclaw (dependency), NON in codice primo-partito.** Vedi §3.
-

2. Buchi rispetto alla demo

1. **Webhook telefonico in ingresso (buco critico — Atti 1 e 5).** Nessun endpoint pubblico, autenticato, generico che riceva una chiamata/messaggio dalla parte telefonica esterna. Ingressi attuali: sync Gmail (email), `/api/crm/events` (eventi app-auth, non conversazione), SNS (solo bounce). Serve un loop `chiamata → agente → risposta` su HTTP: risolvi/crea la Persona dal numero, registra interazione/trascrizione, invoca l'agente con contesto CRM (memoria), restituisce risposta per TTS e/o innesca outbound. **Non negoziabile:** la demo inizia e finisce lì.
2. **Preferenza di contatto + opt-in (Atti 1→3).** Nessun campo su `people` per "Telegram vs email" né consenso marketing. Serve campo enum "Preferred Contact Channel" + boolean "Marketing Opt-in", scritti dal webhook e letti dall'orchestrazione.
3. **Catalogo prodotti + ordini (Atti 2, 3, 5).** Il demo cita Samsung Galaxy, data lancio, data acquisto, stato corriere, possessori del modello precedente, propensione. Non esistono `product/order`: gli acquisti sono solo righe generiche in `interaction` (type `Purchase`). Senza `order` con stato/delivery l'Atto 5 non è sostenibile.
4. **Campagne Telegram-native e routing per canale (Atto 3).** Le campagne sono solo email via SES. Serve: trasporto Telegram nel motore campagne (via send

service del runtime), audience filtrata per `Preferred Contact Channel`, mark per canale.

5. **Identità/memoria cross-canale (Atto 5).** Il webhook deve risolvere la Persona **per numero** e, quando si raccoglie numero/email, collegare la stessa persona alla sorgente Telegram. L'identità risolvibile esiste ma va cablata nel path del telefono.
 6. **Seed/dati demo retail + runbook.** Il seed attuale è generico (5 persone/3 aziende). Serve scenario retail riproducibile (catalogo, date lancio, Lorenzo, ordini con delivery, conversazioni, segmento).
 7. **(Non bloccanti, da `todo.md`)** MCP per integrazioni, ElevenLabs (voce), UX chat (live thinking, model picker), skill docs Apollo/Exa.
-

3. Nota di verifica su Telegram outbound

- `src/infra/outbound/deliver.ts` è uno **stub di tipi per i test** — tutti i canali sono campi `Optional` identici; commento: *"full implementation may live in upstream"*.
 - L'implementazione reale è nel **runtime openclaw** (`node_modules/openclaw`):
 - `extensions/telegram/src/channel.ts` — plugin canale completo (account/bot token, onboarding, pairing, `getChatChannelMeta("telegram")`).
 - `dist/plugin-sdk/infra/outbound/` — sottosistema outbound: `outbound-send-service`, `delivery-queue`, `bound-delivery-router`, `channel-adapters`, `target-resolver`.
 - Conseguenza: **non va costruito l'outbound Telegram**, va **cablate l'invocazione del send service del runtime** dal motore campagne e dal webhook. Per WhatsApp lo stesso lavoro non avrebbe nemmeno il lato canale/credenziali nella dipendenza.
-

4. Piano (ordinato per valore/rischio)

Fase 0 — Contratto webhook con la parte telefonica (giorni 0-1, bloccante)

Bloccare col provider esterno:

- formato payload in ingresso (trascrizione streaming vs a-blocco; id chiamata/numero; metadati);
- sincronia vs asincrona della risposta (attesa TTS inline vs callback);
- chi possiede l'OUTBOUND Telegram (harness via send service del runtime, o parte telefonica).

Assunzione di default: l'harness espone `POST /api/webhooks/phone` (auth con secret/signature) che restituisce la risposta dell'agente per il TTS; l'outbound Telegram è possesso dell'harness.

Fase 1 — Webhook telefonico + identità (sblocca Atti 1, 4, 5)

- Nuovo endpoint `POST /api/webhooks/phone` (e/o `/api/webhooks/telegram`): auth (secret/x-write-key, già esistente), risolvi→crea Persona per numero, scrivi `interaction` di call/conversazione, invoca l'agente `crm` con contesto, ritorna risposta.
- Riutilizza `events.ts / callGatewayRpc` (già in `agent-runner.ts`).
- Test di route sul modello `events/route.test.ts`.

Fase 2 — Campi preferenza + opt-in su **people** + seeding retail

- Migration schema (pattern `workspace-schema-migrations.ts`), scrittura dal webhook, lettura dall'orchestrazione.
- Seed scenario completo (catalogo, Lorenzo, ordini, conversazioni, segmento).

Fase 3 — Modello commerce (product/order) + ingestimento eventi

- Oggetti `product` + `order` (relazioni m2o a people, stato delivery) e ingestimento da webhook/eventi.
- Rende credibili gli Atti 2/5 (propensione, modello precedente, "dov'è il pacco").

Fase 4 — Campagne multicanale

- Estendere `campaigns.ts` per trasporto Telegram (audience filtrata per `Preferred Contact Channel`, invio via send service del runtime), mantenendo SES per email.
- Orchestrazione unica che smista per preferenza (Atto 3).

Fase 5 — Runbook demo + rifiniture presentabilità

- Script della demo atto-per-atto, dati pre-seed; item `todo.md` che contano a schermo (live thinking, model picker) se il tempo lo consente.

5. Decisioni da chiudere prima di implementare

1. **Formato webhook telefonico**: trascrizione a blocchi per turno vs flusso continuo; chi decide il turno.
2. **Outbound Telegram**: orchestrato dall'harness (send service del runtime, bot token) o dalla parte telefonica?
3. **Auth webhook** esterno→harness: secret condiviso (body/header) oppure signature HMAC (consigliata).

6. Conclusione

Il collo di bottiglia è la **Fase 0/1**: senza contratto + webhook telefonico la demo non parte. Il resto (segmenti, copilot, campagne email, canale Telegram via runtime) esiste già e regge. Rispetto a un'ipotesi WhatsApp, Telegram toglie il rischio di costruire il lato canale/credenziali: il runtime openclaw lo fornisce già.

